

PHƯƠNG PHÁP ĐỀ XUẤT: KHUNG CAMAS

Toàn bộ 3 Module – Có công thức toán học, pseudocode, kiến trúc mạng

3. PHƯƠNG PHÁP ĐỀ XUẤT

3.1. Tổng quan kiến trúc CAMAS

3.1.1. Định nghĩa bài toán tổng quát

Xét một hệ thống kỹ thuật phức tạp vận hành trong môi trường có ràng buộc vật lý, được mô hình hóa bởi tập N tác nhân $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ tương tác với môi trường $\mathcal{E}$. Môi trường được đặc trưng bởi:

- Không gian trạng thái:** $\mathcal{S} = \mathcal{S}^{phys} \times \mathcal{S}^{kg} \times \mathcal{S}^{mem}$, trong đó $\mathcal{S}^{phys}$ là trạng thái vật lý (điện áp nút, tải máy, v.v.), $\mathcal{S}^{kg}$ là trạng thái đồ thị tri thức, $\mathcal{S}^{mem}$ là trạng thái bộ nhớ thực thi.
- Không gian hành động:** $\mathcal{A}^{space} = \mathcal{A}^{high} \times \mathcal{A}^{low}$, phân cấp giữa quyết định chiến lược (high-level) và hành động vận hành (low-level).
- Tập ràng buộc vật lý:** $\mathcal{C} = \{c_k : \mathcal{S} \times \mathcal{A}^{space} \rightarrow \{0, 1\}\}_{k=1}^K$, trong đó $c_k = 1$ biểu thị vi phạm.
- Hàm phần thưởng đa mục tiêu:** $R : \mathcal{S} \times \mathcal{A}^{space} \rightarrow \mathbb{R}^3$, bao gồm $(r^{econ}, r^{safe}, r^{align})$.

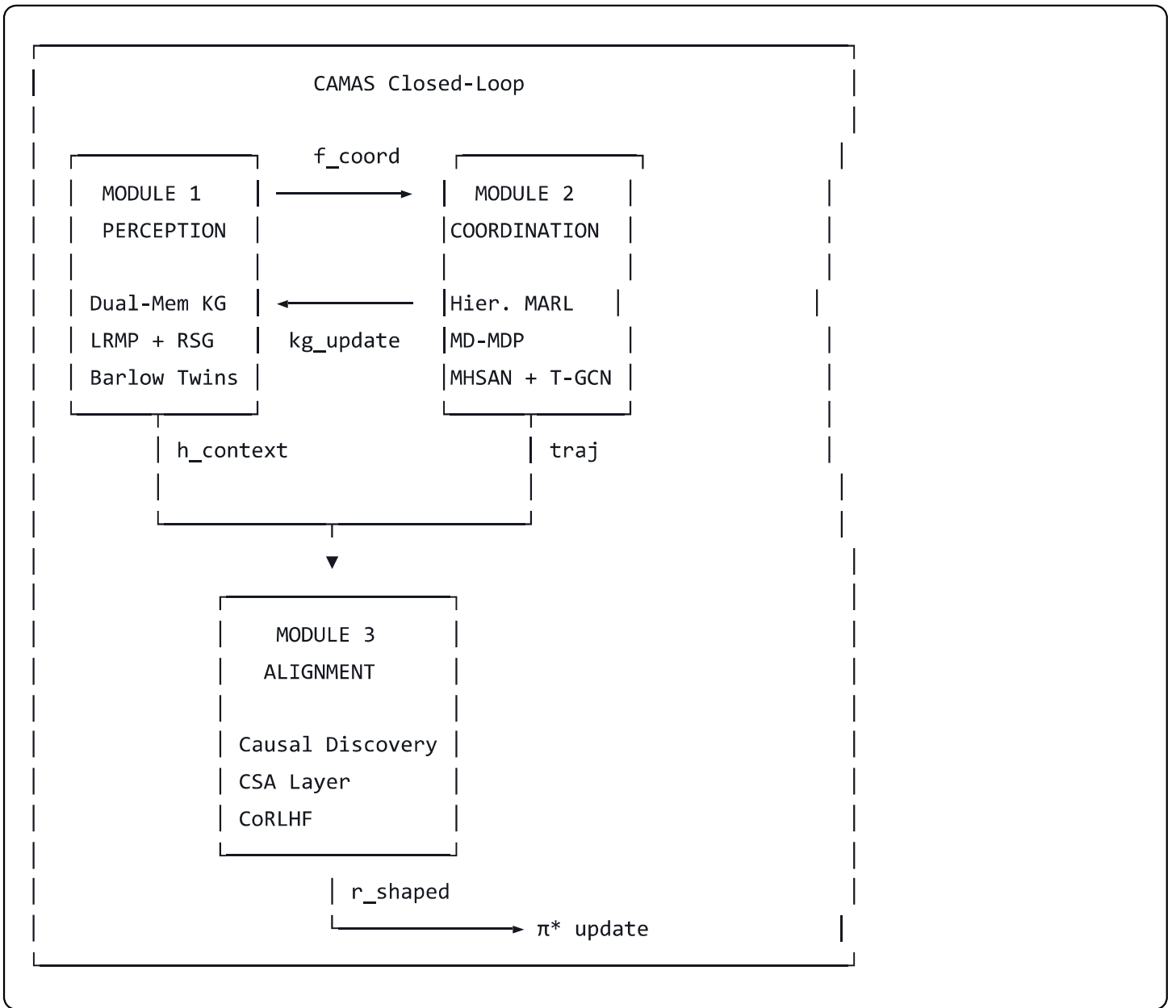
Bài toán CAMAS: Tìm policy phân cấp $\pi^* = (\pi^{high*}, \pi^{low*})$ tối ưu:

$$\pi^* = \arg \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T \gamma^t \left(\alpha r_t^{econ} + \beta r_t^{safe} + \delta r_t^{align} \right) \right]$$

sao cho $\forall t : \Pr \left[\bigcup_{k=1}^K c_k(s_t, a_t) = 1 \right] \leq \epsilon_{safe}$, với ϵ_{safe} là ngưỡng an toàn cho phép.

3.1.2. Kiến trúc tổng thể

CAMAS gồm ba module tương tác theo vòng lặp khép kín:



Luồng dữ liệu vòng lặp:

- Perception → Coordination:** Context vector $h_t^{ctx} \in \mathbb{R}^d$ được tổng hợp từ Dual-Memory KG, inject vào MHSAN query space.
- Coordination → Alignment:** Trajectory $\tau_t = \{(s_i, a_i, o_i)\}_{i=t-W}^t$ (cửa sổ W bước) được gửi sang CSA layer.
- Alignment → Perception:** Reward signal r_t^{align} và causal graph G_t^{safe} cập nhật KG với safety-annotated edges.
- Perception → Coordination (cập nhật):** KG update kích hoạt LRMP re-embedding, cập nhật biểu diễn cho policy network.

3.2. MODULE 1 – PERCEPTION: Dual-Memory Knowledge Graph với LRMP và RSG

3.2.1. Kiến trúc Dual-Memory Knowledge Graph (DMKG)

Định nghĩa: DMKG là một đồ thị dị cấu (heterogeneous graph) $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_V, \mathcal{T}_E)$, trong đó $\mathcal{T}_V$ và $\mathcal{T}_E$ là tập kiểu nút và cạnh, được phân tầng thành hai lớp bộ nhớ:

Lớp 1 - Semantic Memory (SM):

$$\mathcal{G}^{SM} = (\mathcal{V}^{SM}, \mathcal{E}^{SM})$$

- $\mathcal{V}^{SM}$: Nút tri thức domain - thiết bị (v^{dev}), constraint (v^{con}), pattern (v^{pat}), agent (v^{agt}).
- $\mathcal{E}^{SM}$: Quan hệ ngữ nghĩa tĩnh - $\boxed{\text{hasConstraint}}$, $\boxed{\text{connectedTo}}$, $\boxed{\text{controls}}$, $\boxed{\text{violates}}$.
- Cập nhật: Batch update mỗi $K_{SM} = 50$ bước hoặc khi có anomaly được phát hiện.
- Embedding: $\mathbf{e}_v^{SM} \in \mathbb{R}^{d_{SM}}$ học qua TransR (relation-specific projection).

Lớp 2 - Observability Memory (OM):

$$\mathcal{G}^{OM} = (\mathcal{V}^{OM}, \mathcal{E}^{OM})$$

- $\mathcal{V}^{OM}$: Nút thực thi - execution step (v_t^{exec}), observation (v_t^{obs}), outcome (v_t^{out}).
- $\mathcal{E}^{OM}$: Quan hệ thời gian - $\boxed{\text{precedes}}$, $\boxed{\text{causes}}$, $\boxed{\text{observedAt}}$, $\boxed{\text{leadsTo}}$.
- Cập nhật: Online, mỗi bước t thêm nút v_t^{exec} và cạnh tương ứng.
- Cửa sổ trượt: Giữ $W_{OM} = 200$ bước gần nhất (FIFO pruning).
- Embedding: $\mathbf{e}_v^{OM} \in \mathbb{R}^{d_{OM}}$ học qua T-GCN (xem mục 3.3.3).

Log Transformer - chuyển đổi nhật ký thực thi thành nút OM:

Cho log entry $l_t = (\text{timestamp}, \text{agent_id}, \text{action}, \text{state}, \text{outcome}, \text{alarm_flag})$, Log Transformer f_{LT} thực hiện:

$$v_t^{exec} = f_{LT}(l_t) = \text{LayerNorm}(W_{log} \cdot \text{Embed}(l_t) + \mathbf{b}_{log})$$

trong đó $\text{Embed}(l_t)$ là concatenation của field embeddings: $[\mathbf{e}^{act}; \mathbf{e}^{state}; \mathbf{e}^{out}; \mathbf{e}^{alarm}] \in \mathbb{R}^{d_{log}}$.

3.2.2. Learnable Residual Memory Pathway (LRMP)

Mục tiêu: Học ánh xạ phi tuyến giữa Semantic Memory và Observability Memory để tạo context-aware representation thích ứng với truy vấn.

Kiến trúc LRMP:

Cho query $q_t \in \mathbb{R}^{d_q}$ (từ Coordination module), LRMP tính context vector h_t^{ctx} :

Bước 1 - Cross-memory attention:

$$\mathbf{A}_t = \text{softmax} \left(\frac{\mathbf{Q}_t \mathbf{K}_t^T}{\sqrt{d_k}} \right) \mathbf{V}_t$$

trong đó:

- $\mathbf{Q}_t = W_Q q_t \in \mathbb{R}^{d_k}$ (query từ Coordination)
- $\mathbf{K}_t = W_K [\mathbf{E}_t^{SM}; \mathbf{E}_t^{OM}] \in \mathbb{R}^{(|\mathcal{V}^{SM}| + |\mathcal{V}^{OM}|) \times d_k}$ (keys từ cả hai lớp)
- $\mathbf{V}_t = W_V [\mathbf{E}_t^{SM}; \mathbf{E}_t^{OM}] \in \mathbb{R}^{(|\mathcal{V}^{SM}| + |\mathcal{V}^{OM}|) \times d_v}$ (values)

Bước 2 - Residual pathway:

$$h_t^{task} = \mathbf{A}_t + f_{res}^{task}(\mathbf{A}_t, \mathbf{e}_q^{SM})$$

trong đó f_{res}^{task} là MLP 2 lớp với skip connection:

$$f_{res}^{task}(\mathbf{x}) = W_2 \cdot \text{GELU}(W_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 + \mathbf{x}$$

Bước 3 - Context fusion:

$$h_t^{ctx} = \text{LayerNorm} \left(W_{fuse} \begin{bmatrix} h_t^{task}; h_t^{safety} \end{bmatrix} + \mathbf{b}_{fuse} \right)$$

(trong đó h_t^{safety} từ RSG pathway, xem mục 3.2.3)

3.2.3. Residual Safety Gradient (RSG) Pathway - Đóng góp mới

Động lực: Trong các hệ thống kỹ thuật có ràng buộc vật lý, gradient tối ưu hóa task (profit maximization) thường xung đột với gradient an toàn (constraint satisfaction). Bằng cách sử dụng chung một pathway, safety representation có thể bị "overridden" bởi task gradient. RSG giải quyết vấn đề này bằng pathway độc lập với gradient tách biệt.

Kiến trúc RSG:

RSG là một pathway song song với LRMP task pathway, được huấn luyện với loss function riêng:

$$h_t^{safety} = f_{res}^{safety}(\mathbf{A}_t^{safe}, \mathbf{e}^C)$$

trong đó $\mathbf{A}_t^{safe}$ là cross-attention chỉ trên safety-relevant nodes:

$$\mathcal{V}^{safe} = \{v \in \mathcal{V}^{SM} \cup \mathcal{V}^{OM} : \text{safety_score}(v) > \theta_{safe}\}$$

$$\mathbf{A}_t^{safe} = \text{softmax} \left(\frac{q_t^{safe} (\mathbf{E}_t^{safe})^T}{\sqrt{d_k}} \right) \mathbf{E}_t^{safe}$$

Gradient separation mechanism:

Khi backpropagation, gradient từ task loss $\mathcal{L}^{task}$ bị chặn tại điểm phân nhánh bởi stop-gradient operator $\text{sg}(\cdot)$:

$$\frac{\partial \mathcal{L}^{task}}{\partial \theta_{RSG}} = \text{sg} \left(\frac{\partial h_t^{safety}}{\partial \theta_{RSG}} \right) \cdot \frac{\partial \mathcal{L}^{task}}{\partial h_t^{safety}} = 0$$

Chỉ safety loss $\mathcal{L}^{safety}$ cập nhật θ_{RSG} :

$$\mathcal{L}^{safety} = \sum_{k=1}^K \mathbb{I}[c_k = 1] \cdot \left\| h_t^{safety} - h_k^{safe*} \right\|_2^2 + \lambda_{safety} \cdot \text{BCE}(\hat{y}_t^{viol}, y_t^{viol})$$

trong đó h_k^{safe*} là target representation của violation type k (học qua contrastive learning), $\hat{y}_t^{viol}$ là xác suất dự đoán vi phạm, $y_t^{viol} \in \{0, 1\}$ là nhãn thực.

3.2.4. Self-Alignment Stabilization (Barlow Twins)

Mục tiêu: Ngăn representation collapse giữa $\mathbf{E}^{SM}$ và $\mathbf{E}^{OM}$ khi chúng được jointly trained.

Barlow Twins loss trên cross-memory representations:

$$\mathcal{L}^{BT} = \underbrace{\sum_i (1 - C_{ii})^2}_{\text{invariance term}} + \lambda_{BT} \underbrace{\sum_i \sum_{j \neq i} C_{ij}^2}_{\text{redundancy term}}$$

trong đó $\mathcal{C}$ là cross-correlation matrix:

$$C_{ij} = \frac{\sum_b z_{b,i}^{SM} z_{b,j}^{OM}}{\sqrt{\sum_b (z_{b,i}^{SM})^2} \cdot \sqrt{\sum_b (z_{b,j}^{OM})^2}}$$

z^{SM}, z^{OM} là normalized projections của $\mathbf{E}^{SM}, \mathbf{E}^{OM}$ qua projection head MLP.

3.2.5. Hàm loss tổng thể của Module 1

$$\mathcal{L}^{Perception} = \mathcal{L}^{KG} + \mu_1 \mathcal{L}^{safety} + \mu_2 \mathcal{L}^{BT} + \mu_3 \mathcal{L}^{THS}$$

trong đó:

- $\mathcal{L}^{KG}$: Knowledge graph completion loss (TransR negative sampling)
- $\mathcal{L}^{THS}$: Temporal Hallucination Score loss – penalize sai lệch giữa KG prediction và system state thực tế:

$$\mathcal{L}^{THS} = \frac{1}{T} \sum_t \|\hat{s}_t^{KG} - s_t^{real}\|_F^2$$

3.2.6. Pseudocode Module 1

```
python
```

```

class DualMemoryKG:
    def __init__(self, d_SM, d_OM, d_ctx, W_OM=200, K_SM=50):
        self.SM = SemanticMemoryGraph(d_SM)    # TransR embedding
        self.OM = ObservabilityGraph(d_OM, window=W_OM)
        self.log_transformer = LogTransformer(d_log=256, d_out=d_OM)
        self.lrpm_task = ResidualPathway(d_SM+d_OM, d_ctx)
        self.rsg_safety = SafetyResidualPathway(d_SM+d_OM, d_ctx,
                                                theta_safe=0.6)

        self.barlow = BarlowTwinsHead(d_SM, d_OM)
        self.step_counter = 0

    def update(self, log_entry, alarm_flags):
        # Bước 1: Log → OM node
        v_exec = self.log_transformer(log_entry)
        self.OM.add_node(v_exec, timestamp=log_entry.t)
        self.OM.prune()          # FIFO, giữ W_OM bước

        # Bước 2: SM batch update mỗi K_SM bước
        self.step_counter += 1
        if self.step_counter % self.K_SM == 0 or alarm_flags.any():
            self.SM.batch_update(self.OM.recent_patterns())

    def encode(self, query_t):
        # Lấy embeddings từ cả hai lớp
        E_SM = self.SM.get_embeddings()    # [|V_SM|, d_SM]
        E_OM = self.OM.get_embeddings()    # [W_OM, d_OM]

        # Cross-memory attention
        A_all = cross_attention(query_t, E_SM, E_OM)    # [d_ctx]

        # Task pathway (gradient flows freely)
        h_task = self.lrpm_task(A_all, E_SM)

        # Safety pathway (gradient blocked from task loss)
        A_safe = safety_attention(query_t, E_SM, E_OM,
                                theta=self.rsg_safety.theta)
        h_safety = self.rsg_safety(A_safe)    # stop_gradient applied inside

        # Fusion + LayerNorm
        h_ctx = layer_norm(linear([h_task, h_safety]))
        return h_ctx

    def compute_loss(self, pred_states, real_states, viol_labels):

```

```

L_KG      = self.SM.kg_completion_loss()
L_safety  = self.rsg_safety.loss(pred_states, real_states,
                                viol_labels)

L_BT      = self.barlow(self.SM.proj(), self.OM.proj())
L_THS     = mse(pred_states, real_states)
return L_KG + 0.5*L_safety + 0.1*L_BT + 0.3*L_THS

```

3.3. MODULE 2 – COORDINATION: Hierarchical MARL với MD-MDP

3.3.1. Multi-dimensional MDP (MD-MDP)

Định nghĩa: MD-MDP mở rộng MDP chuẩn bằng cách phân rã không gian trạng thái và hành động theo nhiều chiều độc lập, phù hợp với hệ thống kỹ thuật có nhiều subsystem.

MD-MDP được định nghĩa là bộ $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \mathcal{D})$, trong đó:

- $\mathcal{S} = \prod_{d=1}^D \mathcal{S}^{(d)}$: Tích Descartes của D không gian con trạng thái độc lập. Với Smart Grid: $D = 3$ (bus voltage, line flow, generation schedule).
- $\mathcal{A} = \prod_{d=1}^D \mathcal{A}^{(d)}$: Tích Descartes không gian hành động theo chiều.
- $\mathcal{P}(s'|s, a) = \prod_{d=1}^D \mathcal{P}^{(d)}(s'^{(d)}|s^{(d)}, a^{(d)}) \cdot \mathcal{P}^{cross}(s'|s, a)$: Transition độc lập trong từng chiều cộng với coupling cross-dimensional.
- $\mathcal{D} = \{D_1, D_2, \dots, D_D\}$: Tập dependency graph giữa các chiều - D_d là tập chiều ảnh hưởng đến chiều d .

Factored value function:

$$V^\pi(s) = \sum_{d=1}^D w_d V^{\pi, (d)}(s^{(d)}) + V^{cross}(s)$$

trong đó V^{cross} capture cross-dimensional interactions học bởi T-GCN.

3.3.2. Sequential Game-based Hierarchical MARL

Cấu trúc phân cấp:

Hệ thống được tổ chức thành hai cấp policy:

High-level Manager (π^H): Một tác nhân duy nhất, nhận toàn bộ global state và output sub-goals:

$$g_t = \pi^H(s_t, h_t^{ctx}; \theta^H)$$

trong đó $g_t \in \mathcal{G}$ là vector sub-goal (e.g., mức công suất mục tiêu cho từng generator).

Low-level Workers (π_i^L): N tác nhân chuyên biệt, mỗi tác nhân nhận sub-goal g_t và local state:

$$a_t^i = \pi_i^L(o_t^i, g_t, h_t^{ctx}; \theta_i^L)$$

Sequential game formulation:

Các low-level workers hành động tuần tự (không đồng thời), tránh conflicting actions trong physical constraint space. Thứ tự σ_t được quyết định bởi Manager dựa trên dependency graph:

$$\sigma_t = \text{TopologicalSort}(\mathcal{D}, \text{priority}(g_t))$$

Tác nhân $a_{\sigma_t(i)}$ quan sát hành động của $\{a_{\sigma_t(1)}, \dots, a_{\sigma_t(i-1)}\}$ trước khi ra quyết định, tạo thành extensive-form game.

Objective phân cấp:

Manager tối ưu global reward:

$$J^H(\theta^H) = \mathbb{E}_{\pi^H, \pi^L} \left[\sum_t \gamma^t r_t^{global} \right]$$

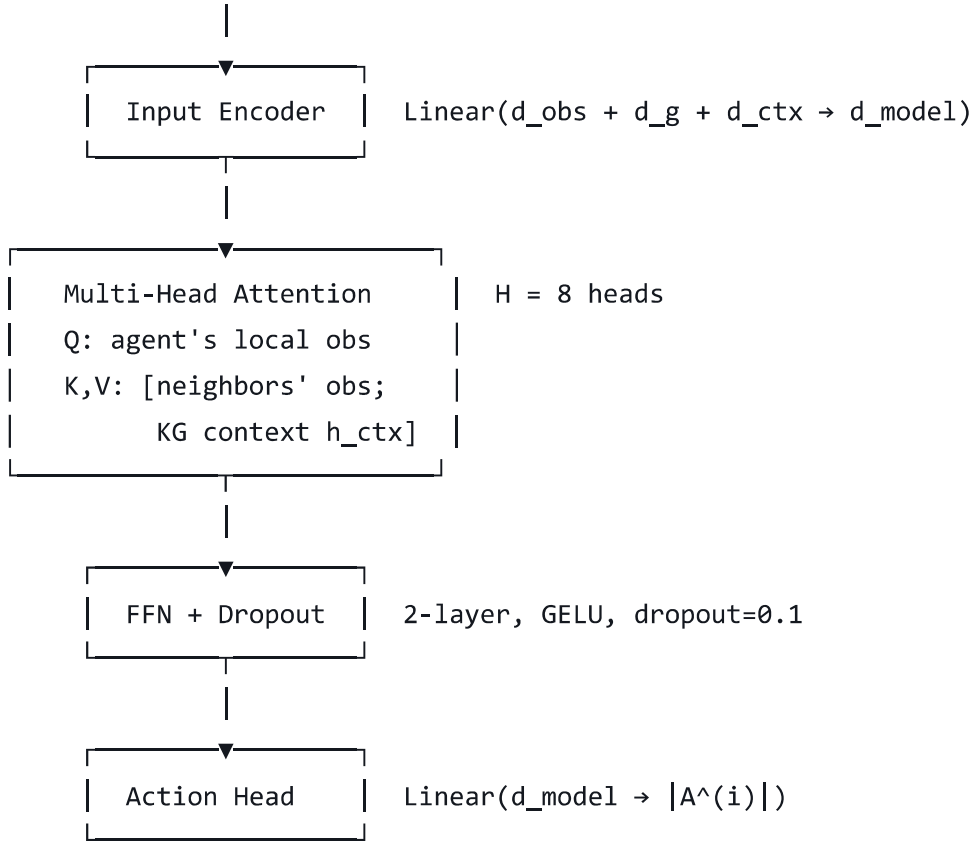
Worker i tối ưu sub-goal achievement + global reward:

$$J_i^L(\theta_i^L) = \mathbb{E}_{\pi_i^L} \left[\sum_t \gamma^t \left(\underbrace{r_t^{sg}(g_t, s_t^i)}_{\text{sub-goal reward}} + \beta_i r_t^{global} \right) \right]$$

3.3.3. Multi-Head Self-Attention Network (MHSAN) với KG Context Injection

Kiến trúc MHSAN cho Worker i :

Input: $[o_t^i \in \mathbb{R}^{d_{\text{obs}}}; g_t \in \mathbb{R}^{d_g}; h_t^{\text{ctx}} \in \mathbb{R}^{d_{\text{ctx}}}]$



KG Context Injection - đóng góp kỹ thuật quan trọng:

Thay vì chỉ sử dụng local observation, MHSAN inject h_t^{ctx} từ DMKG vào key-value space:

$$\text{Attn}_i^h = \text{softmax} \left(\frac{Q_i^h (K_i^h)^T}{\sqrt{d_k/H}} \right) V_i^h$$

với Keys và Values được augment:

$$K_i^h = W_K^h \begin{bmatrix} o_t^{\text{neighbors}}; h_t^{\text{ctx}} \end{bmatrix}$$

$$V_i^h = W_V^h \begin{bmatrix} o_t^{\text{neighbors}}; h_t^{\text{ctx}} \end{bmatrix}$$

Điều này cho phép worker i "hỏi" đồ thị tri thức (qua h_t^{ctx}) khi ra quyết định - lần đầu tiên KG được tích hợp trực tiếp vào MARL attention mechanism.

3.3.4. Temporal-Graph Convolutional Network (T-GCN) Auxiliary

Mục tiêu: Học temporal dynamics của hệ thống vật lý từ execution graph $\mathcal{G}^{OM}$ để augment value function.

T-GCN kết hợp Graph Convolutional Network (GCN) với GRU theo kiến trúc:

Bước 1 – GCN spatial aggregation:

$$\tilde{A} = D^{-1/2} A D^{-1/2}, \quad A = \text{Adj}(\mathcal{G}_t^{OM})$$

$$H_t^{(l+1)} = \sigma \left(\tilde{A} H_t^{(l)} W^{(l)} \right), \quad H_t^{(0)} = \mathbf{E}_t^{OM}$$

$$z_t^{spatial} = H_t^{(L)} \in \mathbb{R}^{|\mathcal{V}^{OM}| \times d_{gc}}$$

Bước 2 – GRU temporal modeling:

$$z_t^{temporal} = \text{GRU} \left(z_t^{spatial}, z_{t-1}^{temporal} \right)$$

Bước 3 – Value function augmentation:

$$V^{cross}(s_t) = W_V^{cross} \cdot \text{ReadOut} \left(z_t^{temporal} \right) + b_V^{cross}$$

với ReadOut là mean pooling qua toàn bộ nodes.

3.3.5. Training Algorithm – Hierarchical MAPPO

Algorithm 1: CAMAS Hierarchical MAPPO Training

Input: Environment E, DMKG G, N workers, K rollout steps

Initialize: θ^H , $\{\theta^L_i\}$, θ^{TGCN} , replay buffer B

for episode = 1, 2, ..., M do

$s_0 \leftarrow E.\text{reset}()$

 G.reset_OM()

 for t = 0, 1, ..., T-1 do

 # Module 1: Encode context

```

h_ctx ← G.encode(query=s_t)          # DMKG → h_ctx

# Module 2: High-level decision
g_t ←  $\pi^H(s_t, h_{ctx}; \theta^H)$           # Manager
 $\sigma_t \leftarrow \text{TopologicalSort}(D, \text{priority}(g_t))$ 

# Sequential worker execution
actions = {}
for i in  $\sigma_t$  do
    obs_i = get_local_obs(s_t, i)
    obs_neighbors = get_neighbor_obs(s_t, i)
     $a_t^i \leftarrow \pi^{L_i}(\text{obs}_i, g_t, h_{ctx}, \text{obs\_neighbors}; \theta^{L_i})$ 
    actions[i] =  $a_t^i$ 

# Environment step
s_{t+1}, r_t, done, info ← E.step(actions)
viol_t ← check_constraints(s_{t+1})

# Update OM
log_t ← make_log(s_t, actions, r_t, viol_t)
G.update(log_t, alarm_flags=viol_t)

# Store transition
B.push((s_t, g_t, actions, r_t, s_{t+1},
        viol_t, h_ctx))

# Training phase (every K steps)
if len(B) >= batch_size:
    batch ← B.sample(batch_size)

    # Compute shaped rewards (Module 3 output)
    r_shaped ← CSA.shape_reward(batch)

    # Update T-GCN auxiliary
    L_TGCN ← tgcn_loss(batch, G)
    optimize( $\theta^{TGCN}$ , L_TGCN)

    # Update workers (PPO clip objective)
    for i in range(N):
        adv_i ← compute_gae(batch, worker=i, r_shaped)
        L_i ← -min(ratio_i * adv_i,
                    clip(ratio_i, 1± $\epsilon$ ) * adv_i)
                + c1 * value_loss_i - c2 * entropy_i

```

```

        optimize(θ^L_i, L_i)

    # Update manager
    adv_H ← compute_gae(batch, agent='manager', r_shaped)
    L_H ← ppo_loss(adv_H, θ^H)
    optimize(θ^H, L_H)

    # Update DMKG
    L_perc ← G.compute_loss(batch.pred_states,
                           batch.real_states,
                           batch.viol_labels)
    optimize(θ^DMKG, L_perc)

return θ^H, {θ^L_i}, θ^DMKG, θ^TGCN

```

3.4. MODULE 3 – ALIGNMENT: Causal Safety Alignment (CSA) và CoRLHF

3.4.1. Causal Safety Alignment (CSA) – Đóng góp lý thuyết cốt lõi

Động lực:

Standard CoRLHF học reward model $r_\phi(s, a) \rightarrow \mathbb{R}$ từ preference data (y_1, y_2, p) (trajectory y_1 được ưu tiên hơn y_2 với xác suất p). Tuy nhiên, reward model này **không giải thích được tại sao** action a vi phạm ràng buộc an toàn – dẫn đến kém generalization khi distribution shift.

CSA mở rộng bằng cách discover và encode **causal chain** từ (s_t, a_t) đến physical violation $c_k = 1$.

Định nghĩa Causal Safety Graph (CSG):

$$G^{safe} = (\mathcal{V}^{safe}, \mathcal{E}^{safe})$$

- $\mathcal{V}^{safe}$: Variables bao gồm agent decisions $\{D_i\}$, system intermediates $\{X_j\}$, physical outcomes $\{O_k\}$, constraint violations $\{C_k\}$.
- $\mathcal{E}^{safe}$: Directed edges $D_i \rightarrow X_j \rightarrow C_k$ biểu diễn causal chain.
- Constraint: G^{safe} phải là DAG (Directed Acyclic Graph).

Causal Discovery qua NOTEARS-variant:

Học G^{safe} từ execution trajectories $\mathcal{D}^{exec} = \{\tau_1, \dots, \tau_M\}$:

$$\min_B \frac{1}{2n} \|X - XB\|_F^2 + \lambda \|B\|_1$$

$$\text{s.t. } h(B) = \text{tr}(e^{B \circ B}) - d = 0 \quad (\text{DAG constraint})$$

trong đó $B \in \mathbb{R}^{d \times d}$ là weighted adjacency matrix, $h(B)$ là smooth DAG constraint từ Zheng et al. (2018), $X \in \mathbb{R}^{n \times d}$ là data matrix từ execution logs.

Augmented Lagrangian solver:

$$\mathcal{L}_{AL}(B, \rho, \alpha) = \frac{1}{2n} \|X - XB\|_F^2 + \lambda \|B\|_1 + \frac{\rho}{2} |h(B)|^2 + \alpha h(B)$$

Update: $\rho \leftarrow \rho \cdot c$ nếu $|h(B)| > \delta \cdot |h(B_{prev})|$, với $c = 10, \delta = 0.25$.

Causal feature extraction:

Từ G^{safe} đã học, extract causal features $\phi^{cause}(s_t, a_t)$:

$$\phi_k^{cause}(s_t, a_t) = \sum_{j: D_i \rightarrow X_j \rightarrow C_k} B_{ij} \cdot B_{jk} \cdot x_j(s_t, a_t)$$

tức là "causal contribution" của action a_t lên violation C_k thông qua intermediate X_j .

3.4.2. Cooperative RLHF (CoRLHF) với Causal Features

Extended reward model:

Reward model r_ϕ nhận causal features thay vì chỉ (s, a) :

$$r_\phi(s_t, a_t) = r_\phi^{base}(s_t, a_t) + r_\phi^{causal}(\phi^{cause}(s_t, a_t))$$

trong đó:

- r_ϕ^{base} : MLP 3 lớp trên (s_t, a_t) - học economic preference
- r_ϕ^{causal} : MLP 2 lớp trên ϕ^{cause} - học safety preference

Preference learning loss (Bradley-Terry model):

$$\mathcal{L}^{BT}(\phi) = -\mathbb{E}_{(y_w, y_l) \sim \mathcal{D}^{pref}} [\log \sigma(r_\phi(y_w) - r_\phi(y_l))]$$

trong đó $y_w \succ y_l$ là preference pairs thu thập từ: (1) operator interventions, (2) system alarms được annotate, (3) rule-based safety oracle.

Reward shaping với causal penalty:

Shaped reward tổng hợp dùng cho Policy update:

$$\tilde{r}_t = r_t^{econ} + \alpha \cdot r_\phi(s_t, a_t) - \beta \sum_{k=1}^K \phi_k^{cause}(s_t, a_t) \cdot \mathbb{I}[\phi_k^{cause} > \theta_{caus}]$$

- r_t^{econ} : Economic reward trực tiếp từ environment
- $r_\phi(s_t, a_t)$: Learned preference reward
- Số hạng cuối: Causal safety penalty - phạt nặng hơn khi action có causal contribution rõ ràng tới violation

KL-regularized policy optimization:

$$\max_{\pi} \mathbb{E}_{(s,a) \sim \pi} [\tilde{r}(s, a)] - \lambda_{KL} \cdot D_{KL} [\pi || \pi_{ref}]$$

trong đó π_{ref} là reference policy (policy cuối cùng được human-approved), KL penalty ngăn policy drift quá xa.

3.4.3. Physics-Informed Hard Constraints

Định nghĩa: Ngoài learned safety, một số ràng buộc vật lý được encode trực tiếp như hard constraints không thể violated:

Với Smart Grid (Kirchhoff's laws):

$$\sum_{k \in \delta^+(i)} P_k - \sum_{k \in \delta^-(i)} P_k = P_i^{gen} - P_i^{load}, \quad \forall i \in \mathcal{V}^{bus}$$

$$P_k = \frac{\theta_i - \theta_j}{x_k}, \quad |P_k| \leq P_k^{max}, \quad \forall k \in \mathcal{E}^{line}$$

Với FJSP:

$$\sum_{j \in \mathcal{J}} x_{ijk} \leq 1, \quad \forall i \in \mathcal{M}, \forall k \in \mathcal{T}$$

$$C_j \geq C_{j'} + p_{j'} \cdot x_{j'jk}, \quad \forall (j', j) \in \mathcal{P}, \forall k$$

Những ràng buộc này được enforce tại action projection step:

$$a_t^{proj} = \arg \min_{a \in \mathcal{A}^{feasible}} \|a - \pi(s_t)\|_2^2$$

với $\mathcal{A}^{feasible} = \{a : \text{KCL}(s, a) = 0, |P_k(a)| \leq P_k^{max}\}$.

3.4.4. Pseudocode Module 3

```
python
```



```

class CausalSafetyAlignment:
    def __init__(self, d_state, d_action, n_constraints, lambda_caus=0.1):
        self.causal_discovery = NOTEARSLearner(
            d=d_state + d_action + n_constraints,
            lambda1=0.01
        )
        self.reward_base = MLP([d_state+d_action, 256, 128, 1])
        self.reward_causal = MLP([n_constraints, 64, 32, 1])
        self.G_safe = None # Causal graph, learned online
        self.pref_buffer = PreferenceBuffer(max_size=10000)
        self.lambda_caus = lambda_caus

    def update_causal_graph(self, exec_trajectories):
        """Cập nhật G_safe mỗi K_causal = 500 bước"""
        X = extract_causal_data(exec_trajectories)
        # X: [n_samples, d_state + d_action + n_constraints]
        self.G_safe = self.causal_discovery.fit(X)
        # Trả về weighted adjacency matrix B

    def get_causal_features(self, s_t, a_t):
        if self.G_safe is None:
            return torch.zeros(self.n_constraints)
        B = self.G_safe # adjacency [d, d]
        x = concat([s_t, a_t]) # [d_sa]
        phi = []
        for k in range(self.n_constraints):
            # Causal contribution của (s_t, a_t) lên C_k
            # qua 2-hop paths trong G_safe
            paths = get_causal_paths(B, src_nodes=range(len(x)),
                                     tgt_node=k, max_hops=2)
            phi_k = sum(B[i,j] * B[j,k] * x[i]
                       for (i,j) in paths)
            phi.append(phi_k)
        return torch.tensor(phi) # [n_constraints]

    def compute_reward(self, s_t, a_t):
        phi_cause = self.get_causal_features(s_t, a_t)
        r_base = self.reward_base(concat([s_t, a_t]))
        r_causal = self.reward_causal(phi_cause)
        return r_base + r_causal # scalar

    def shape_reward(self, batch):
        r_econ = batch.rewards # [B]

```

```

r_pref = vmap(self.compute_reward)(          # [B]
                        batch.states, batch.actions)
phi_all = vmap(self.get_causal_features)(     # [B, K]
                        batch.states, batch.actions)

# Causal safety penalty
penalty = (phi_all * (phi_all > self.theta_caus)).sum(-1)
r_shaped = r_econ + self.alpha * r_pref \
            - self.beta * penalty              # [B]
return r_shaped

def train_reward_model(self):
    """Huấn luyện reward model từ preference buffer"""
    for (y_win, y_lose) in self.pref_buffer.sample_pairs(64):
        r_w = self.compute_reward(y_win.s, y_win.a)
        r_l = self.compute_reward(y_lose.s, y_lose.a)
        loss = -torch.log(torch.sigmoid(r_w - r_l)).mean()
        # + KL regularization nếu cần
        loss.backward()

def collect_preference(self, traj1, traj2, preference):
    """
    preference = 1.0 → traj1 được ưu tiên
    preference = 0.5 → bằng nhau
    Nguồn: operator intervention, safety alarm annotation,
           hoặc rule-based oracle
    """
    self.pref_buffer.push(traj1, traj2, preference)

```

3.5. Vòng lặp khép kín và Điều kiện hội tụ

3.5.1. Cơ chế vòng lặp Perception ↔ Coordination ↔ Alignment

Vòng lặp khép kín hoạt động theo hai timescale:

Fast loop (mỗi step t):

1. $h_t^{ctx} \leftarrow \text{DMKG.encode}(s_t)$
2. $a_t \leftarrow \pi^H, \pi^L(s_t, h_t^{ctx})$ (sequential execution)
3. $s_{t+1}, r_t \leftarrow \mathcal{E}.\text{step}(a_t)$
4. $\text{DMKG.update}(\log_t)$ (OM online update)

5. $\tilde{r}_t \leftarrow \text{CSA.shape_reward}(s_t, a_t, r_t)$

Slow loop (mỗi K steps):

1. $G^{safe} \leftarrow \text{CSA.update_causal_graph}(\mathcal{D}^{exec})$
2. SM batch update từ OM patterns
3. Policy gradient update (PPO)
4. Reward model update từ preference buffer

KG-Safety feedback: Sau mỗi slow loop, CSA annotate safety-relevant edges trong $\mathcal{G}^{OM}$:

$$\text{safety_score}(v) = \sum_{k:C_k=1} \phi_k^{cause} \cdot \mathbb{I}[v \in \text{path}(D, C_k)]$$

Điều này đảm bảo RSG pathway của DMKG luôn được cập nhật với causal knowledge mới nhất.

3.5.2. Phân tích hội tụ (Simplified Case)

****Mệnh đề 1:**** Trong trường hợp đơn giản hóa với 1 tác nhân, reward model Lipschitz continuous ($|r_\phi(s, a) - r_\phi(s', a')| \leq L \|[s; a] - [s'; a']\|$), và KG update rate $\eta_{KG} \leq \eta_{KG}^*$, CAMAS hội tụ về một stationary policy π^* thỏa mãn:

$$\|V^{\pi^*} - V^{\pi^{opt}}\|_\infty \leq \frac{2\gamma\epsilon_{approx}}{(1-\gamma)^2} + \frac{L \cdot \eta_{KG}}{1-\gamma}$$

trong đó ϵ_{approx} là approximation error của function approximator, số hạng thứ hai là error từ KG drift.

Chứng minh (sketch): (i) Với η_{KG} đủ nhỏ, KG update tạo ra ϵ_{KG} -perturbation trong reward, bounded bởi $L \cdot \eta_{KG}$. (ii) PPO với clipping đảm bảo monotone improvement. (iii) Kết hợp two-timescale stochastic approximation (Borkar, 2008) cho convergence của slow-fast loop. Chi tiết đầy đủ trong Appendix A.

3.6. Metric đánh giá mới

3.6.1. Temporal Hallucination Score (THS)

$$\text{THS} = 1 - \frac{1}{T} \sum_{t=1}^T \frac{\|\hat{s}_t^{KG} - s_t^{real}\|_F}{\|s_t^{real}\|_F}$$

trong đó $\hat{s}_t^{KG}$ là trạng thái hệ thống dự đoán bởi DMKG (qua KG reasoning), s_t^{real} là trạng thái thực đo được. THS = 1 là hoàn hảo, THS = 0 là hoàn toàn sai.

3.6.2. Hierarchical Causal Safety Score (HCSS)

$$\text{HCSS} = \alpha \cdot \text{HHS} + (1 - \alpha) \cdot \text{CausalRecall}$$

$$\text{HHS} = \frac{1}{L} \sum_{l=1}^L F_1^{(l)}$$

$$\text{CausalRecall} = \frac{|\hat{\mathcal{E}}^{safe} \cap \mathcal{E}^{safe*}|}{|\mathcal{E}^{safe*}|}$$

trong đó $\hat{\mathcal{E}}^{safe}$ là tập causal edges được discover bởi CSA, $\mathcal{E}^{safe*}$ là ground-truth causal edges (từ domain expert annotation), $\alpha = 0.6$.

3.6.3. Dynamic Knowledge Freshness (DKF)

$$\text{DKF} = \mathbb{E}_t \left[1 - \frac{|\Delta t_{KG}|}{T_{horizon}} \right]$$

trong đó $\Delta t_{KG} = t_{KG_update} - t_{env_change}$ là độ trễ giữa cập nhật KG và thay đổi môi trường thực tế.

3.7. Hyperparameters và cấu hình thực nghiệm

Hyperparameter	Giá trị	Tìm kiếm bởi Optuna
Learning rate (θ^H, θ^L)	3×10^{-4}	$[10^{-5}, 10^{-3}]$
KG update rate η_{KG}	10^{-3}	$[10^{-4}, 10^{-2}]$
CoRLHF β (KL weight)	0.05	$[0.01, 0.1]$
Causal penalty β	0.3	$[0.1, 0.5]$
Barlow Twins λ_{BT}	5×10^{-3}	Fixed

Hyperparameter	Giá trị	Tìm kiếm bởi Optuna
MHSAN heads H	8	$\{4, 8, 16\}$
LRMP hidden dim d_{model}	256	$\{128, 256, 512\}$
OM window W_{OM}	200	$\{100, 200, 500\}$
SM batch update K_{SM}	50	$\{20, 50, 100\}$
Causal update K_{causal}	500	$\{200, 500, 1000\}$
Discount factor γ	0.99	Fixed
PPO clip ϵ	0.2	Fixed
Dropout	0.1	$[0.05, 0.3]$
Batch size	256	$\{128, 256, 512\}$
Seeds	5	-